

操作系统

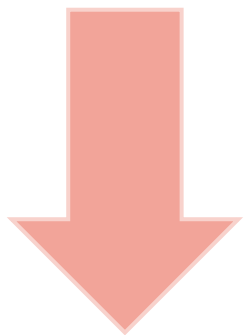
Principles of Operating System

北京大学计算机科学技术系 陈向群

Department of computer science and
Technology, Peking University

2026 Spring

从进程线程的特征出发



并发是所有问题的基础
并发是操作系统设计的基础

并发

- 执行过程不是连续的，是间断性的
- 相对执行速度不可预测

共享

- 进程/线程之间的制约性

不确定性

- 执行的结果与其执行的相对速度有关，
是不确定的

问题

- ▶ 并发环境下程序的执行是否会受到进程/线程调度结果的影响？
- ▶ 举例说明进程/线程互斥、进程/线程同步的场景及相关问题
- ▶ 信号量及PV操作是否可以既解决同步问题又解决互斥问题？
- ▶ 锁的实现有哪些方法？
- ▶ 条件变量及相应的操作解决的是什么问题？
- ▶ 管程机制的主要实现特点是什么？
- ▶ Pthread中解决同步互斥对应的API有哪些？

本章要求掌握的概念

临界区

进程/线程互斥

进程/线程同步

信号量

P、V操作

管程

条件变量

wait/signal

锁

Hoare管程

Mesa管程

Pthreads

生产者消费者问题

读者写者问题

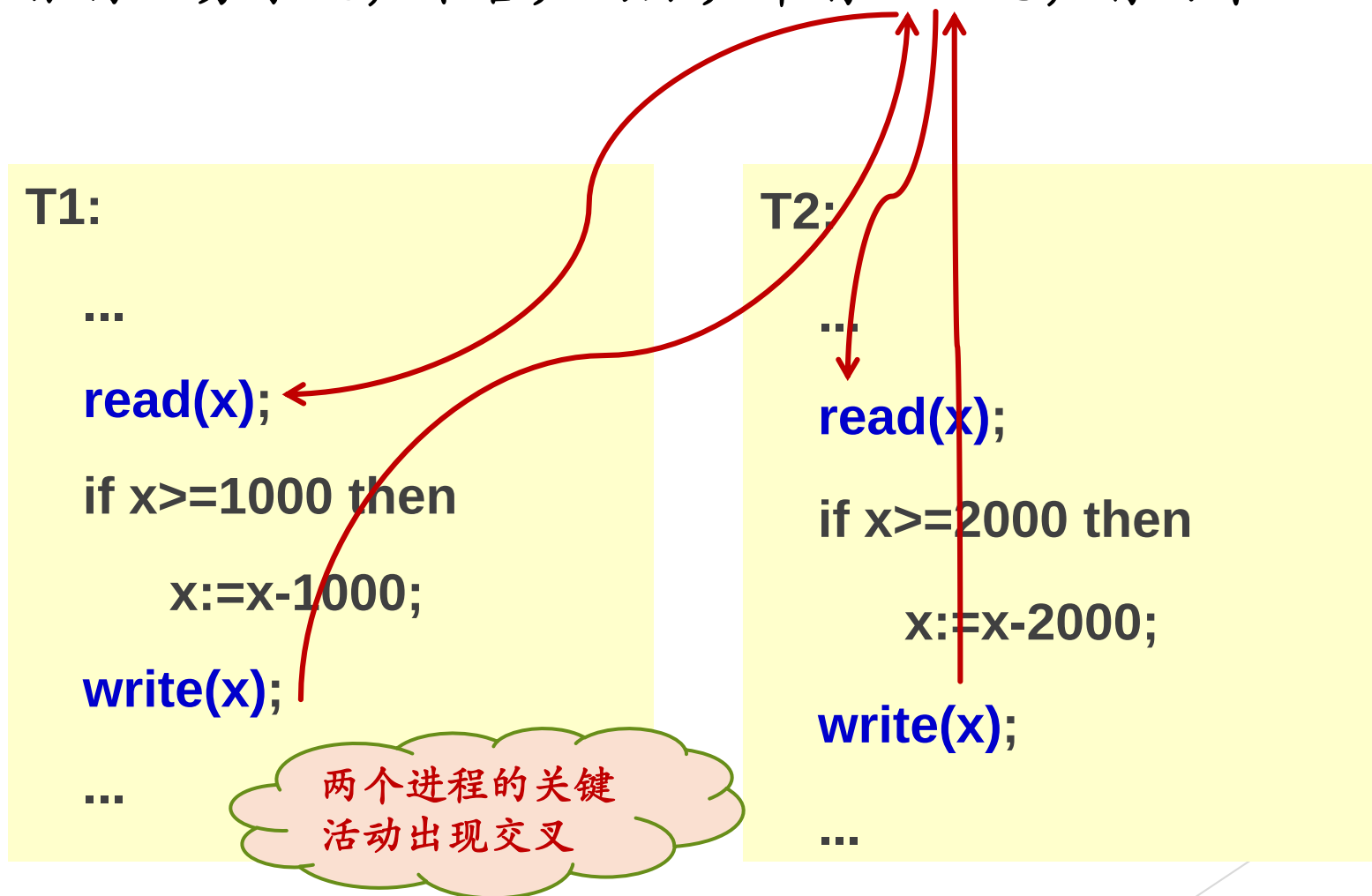
...4...

进程/线程的并发执行

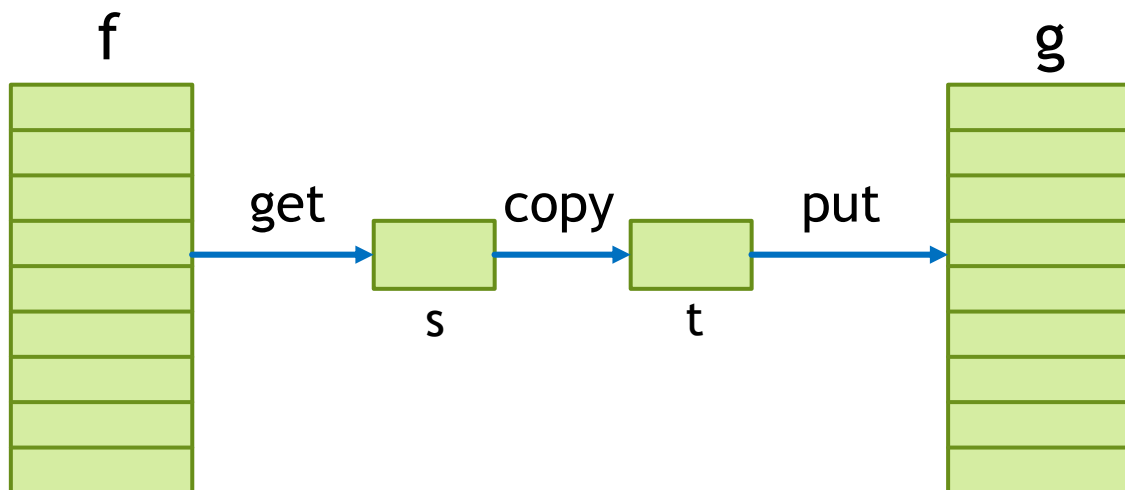
并发环境下进程/线程执行会遇到什么问题？

与时间有关的错误——例子1

某银行业务系统，某客户的账户中有5000元，有两个ATM机T1和T2



与时间有关的错误——例子2

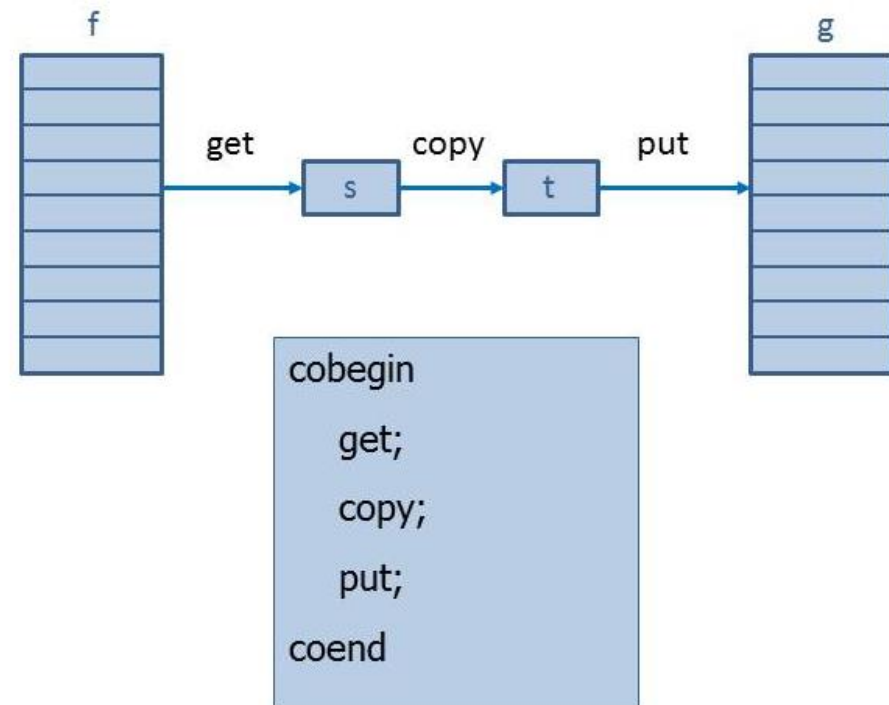


场景

```
cobegin  
  get;  
  copy;  
  put;  
coend
```

get、copy和put三个进程并发执行

并发执行过程分析



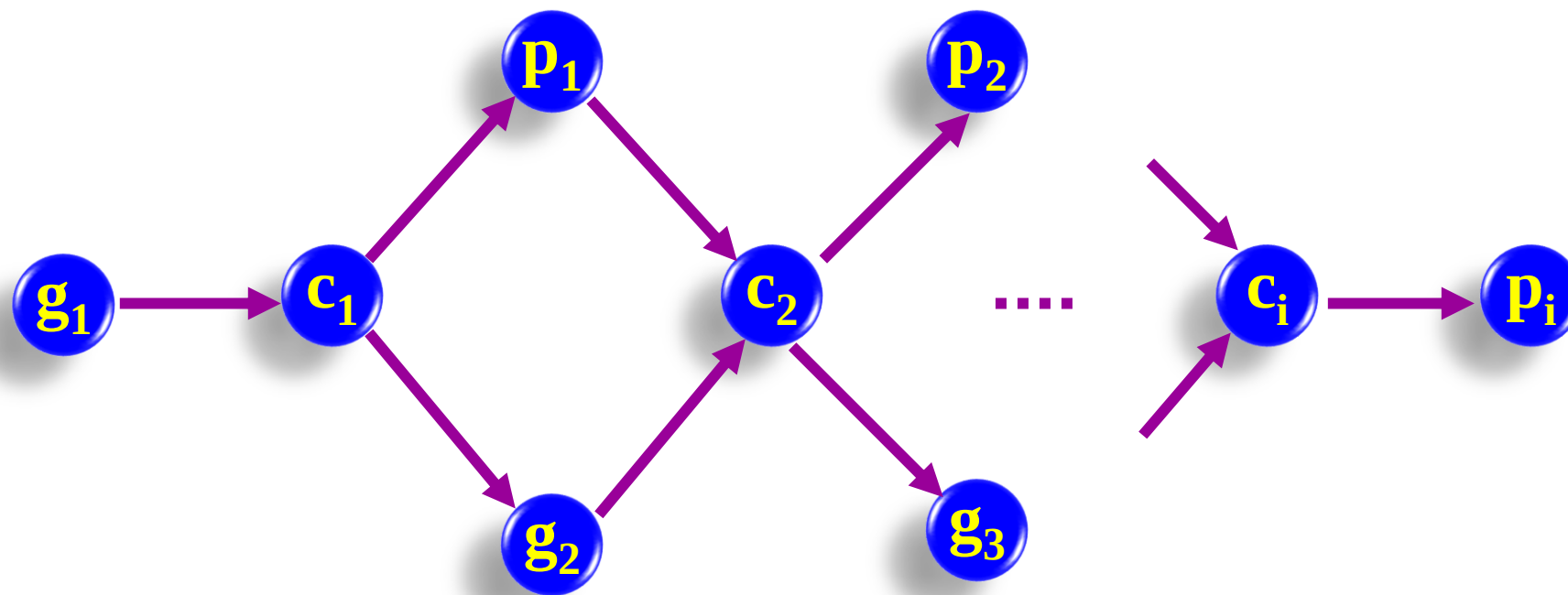
当前状态 *f* *s* *t* *g*
(3,4,...,m) 2 2 (1,2)

可能的执行（假设*g,c,p*为*get,copy,put*的一次循环）

<i>g,c,p</i>	(4,5,...,m)	3	3	(1,2,3) ✓
<i>g,p,c</i>	(4,5,...,m)	3	3	(1,2,2) ✗
<i>c,g,p</i>	(4,5,...,m)	3	2	(1,2,2) ✗
<i>c,p,g</i>	(4,5,...,m)	3	2	(1,2,2) ✗
<i>p,c,g</i>	(4,5,...,m)	3	2	(1,2,2) ✗
<i>p,g,c</i>	(4,5,...,m)	3	3	(1,2,2) ✗

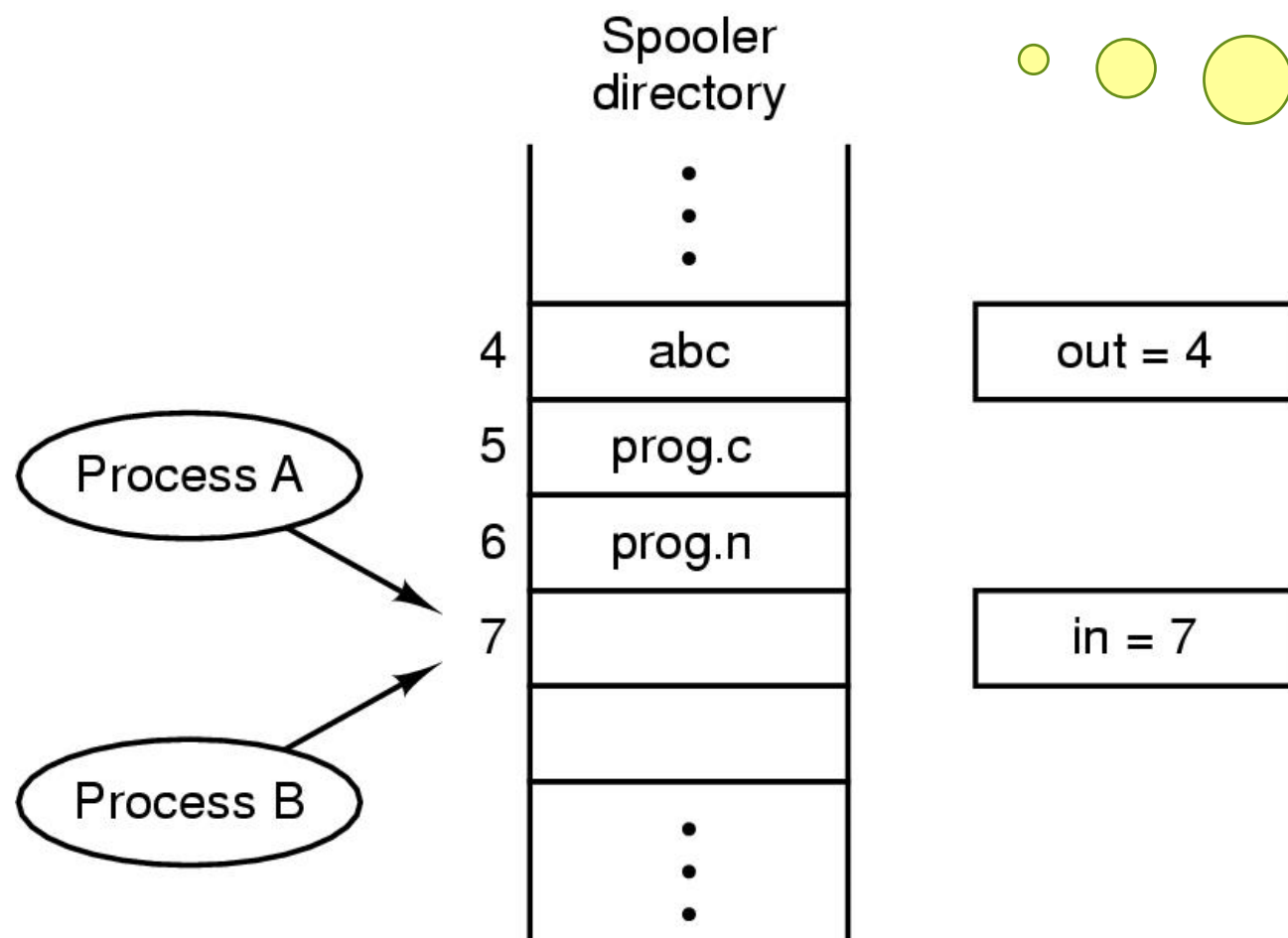
设信息长度为*m*，有多少种可能性？

前趋图



并发环境下程序间的制约关系

竞争条件 (race condition)



两个或多个进程
读写某些共享数
据，而最后的结
果取决于进程运
行的精确时序

Assembly Code for Counter Loop

C code for counter loop in thread i

```
for (i = 0; i < niters; i++)  
    cnt++;
```

Asm code for thread i

```
movq  (%rdi), %rcx  
testq %rcx,%rcx  
jle   .L2  
movl  $0, %eax
```

} H_i : Head

.L3:

```
movq  cnt(%rip), %rdx  
addq  $1, %rdx  
movq  %rdx, cnt(%rip)
```

} L_i : Load cnt
 U_i : Update cnt
 S_i : Store cnt

```
addq  $1, %rax  
cmpq  %rcx, %rax  
jne   .L3
```

} T_i : Tail

.L2:

ICS课相
关内容

Concurrent Execution

- ▶ **Key idea:** In general, any sequentially consistent interleaving is possible, but some give an unexpected result!
 - ▶ I_i denotes that thread i executes instruction I
 - ▶ $\%rdx_i$ is the content of $\%rdx$ in thread i 's context

ICS课相
关内容

i (thread)	$instr_i$	$\%rdx_1$	$\%rdx_2$	cnt
1	H_1	-	-	0
1	L_1	0	-	0
1	U_1	1	-	0
1	S_1	1	-	1
2	H_2	-	-	1
2	L_2	-	1	1
2	U_2	-	2	1
2	S_2	-	2	2
2	T_2	-	2	2
1	T_1	1	-	2

OK

Concurrent Execution

- ▶ **Key idea:** In general, any sequentially consistent interleaving is possible, but some give an unexpected result!
 - ▶ I_i denotes that thread i executes instruction I
 - ▶ $\%rdx_i$ is the content of $\%rdx$ in thread i 's context

ICS课相
关内容

i (thread)	$instr_i$	$\%rdx_1$	$\%rdx_2$	cnt
1	H_1	-	-	0
1	L_1	0	-	0
1	U_1	1	-	0
1	S_1	1	-	1
2	H_2	-	-	1
2	L_2	-	1	1
2	U_2	-	2	1
2	S_2	-	2	2
2	T_2	-	2	2
1	T_1	1	-	2



Thread 1
critical section



Thread 2
critical section

OK

Concurrent Execution (cont)

- Incorrect ordering: two threads increment the counter, but the result is 1 instead of 2

i (thread)	instr _i	%rdx ₁	%rdx ₂	cnt
1	H ₁	-	-	0
1	L ₁	0	-	0
1	U ₁	1	-	0
2	H ₂	-	-	0
2	L ₂	-	0	0
1	S ₁	1	-	1
1	T ₁	1	-	1
2	U ₂	-	1	1
2	S ₂	-	1	1
2	T ₂	-	1	1

ICS课相
关内容

Oops!

Concurrent Execution (cont)

► How about this ordering?

i (thread)	instr _i	%rdx ₁	%rdx ₂	cnt
1	H ₁			0
1	L ₁	0		
2	H ₂			
2	L ₂		0	
2	U ₂		1	
2	S ₂		1	1
1	U ₁	1		
1	S ₁	1		1
1	T ₁			1
2	T ₂			1

ICS课相
关内容

Oops!

► We can analyze the behavior using a *progress graph*

线程共享地址空间

线程与进程的主要区别

- ▶ 线程之间的上下文切换时，地址空间保持不变
- ▶ 每个线程有自己的栈

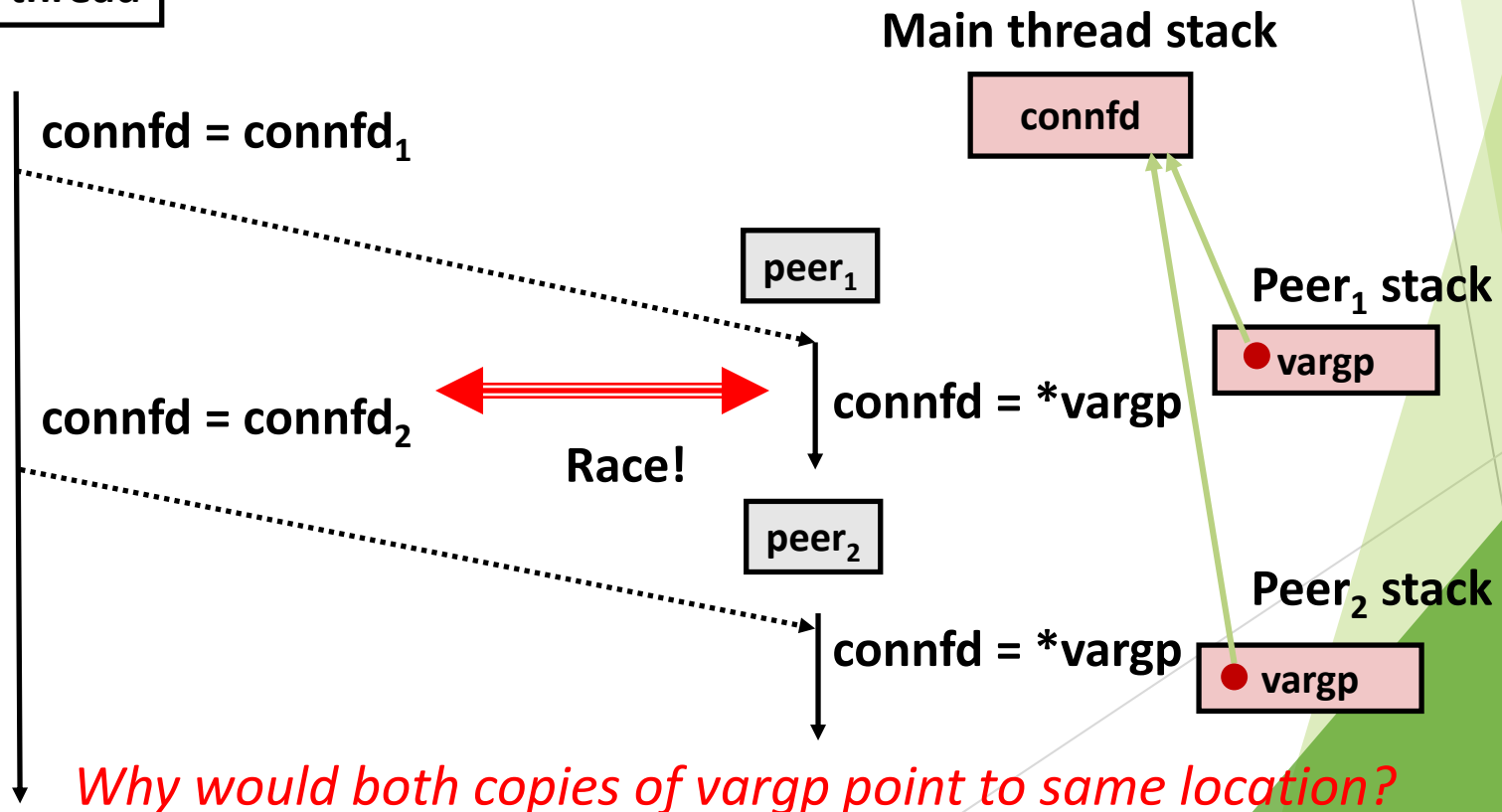
在多线程进程中，每个线程独立运行，每个线程有一个栈

Potential Form of Unintended Sharing

```
while (1) {  
    int connfd = Accept(listenfd, (SA *) &clientaddr, &clientlen);  
    Pthread_create(&tid, NULL, thread, &connfd);  
}
```

main thread

ICS课相
关内容



Could this race occur?这种竞态条件是否会发生?

Main

```
int i;
for (i = 0; i < 100; i++) {
    Pthread_create(&tid, NULL,
                  thread, &i);
}
```

Thread

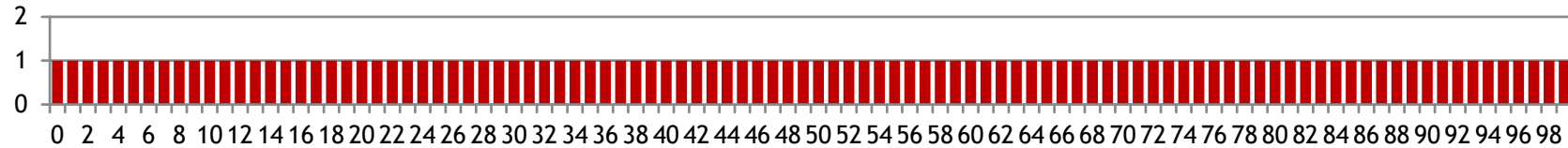
```
void *thread(void *vargp)
{
    int i = *((int *)vargp);
    Pthread_detach(pthread_self());
    save_value(i);
    return NULL;
}
```

► Race Test

- If no race, then each thread would get different value of i
- Set of saved values would consist of one copy each of 0 through 99

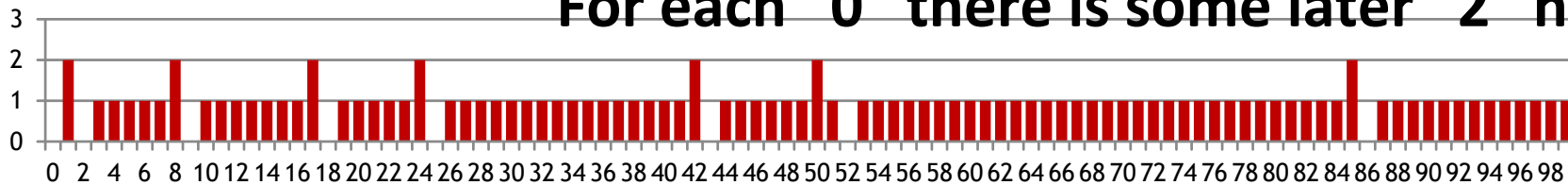
Experimental Results (实验结果)

No Race



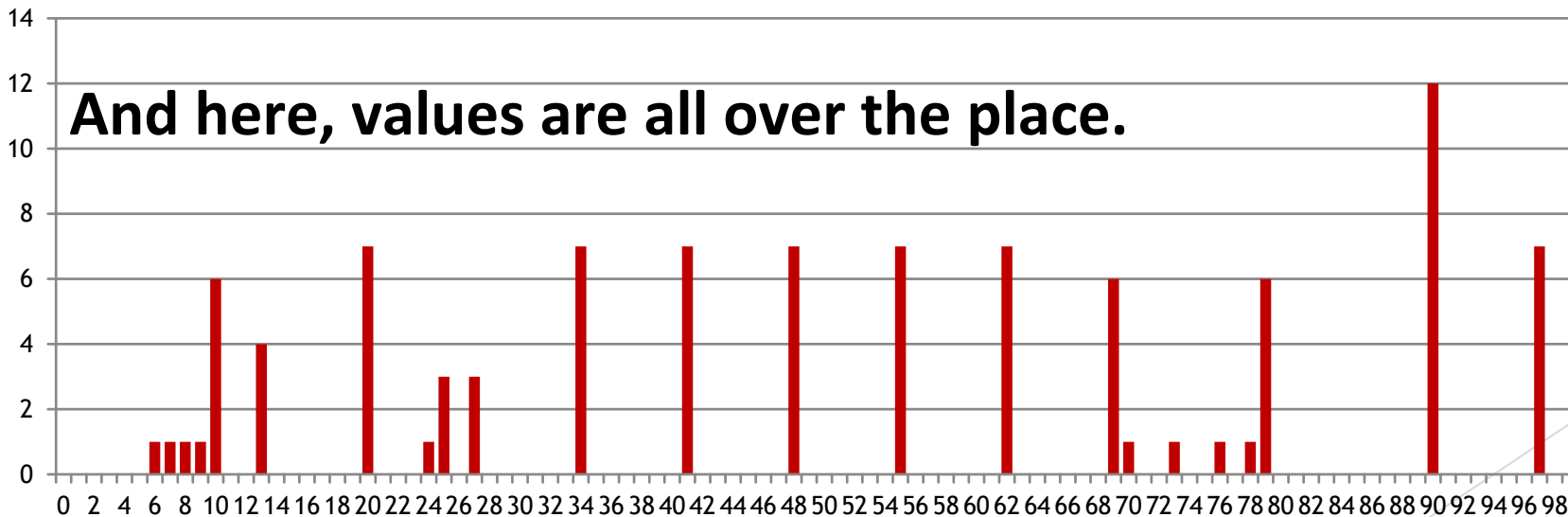
Single core laptop

For each “0” there is some later “2” here



Multicore server

And here, values are all over the place.



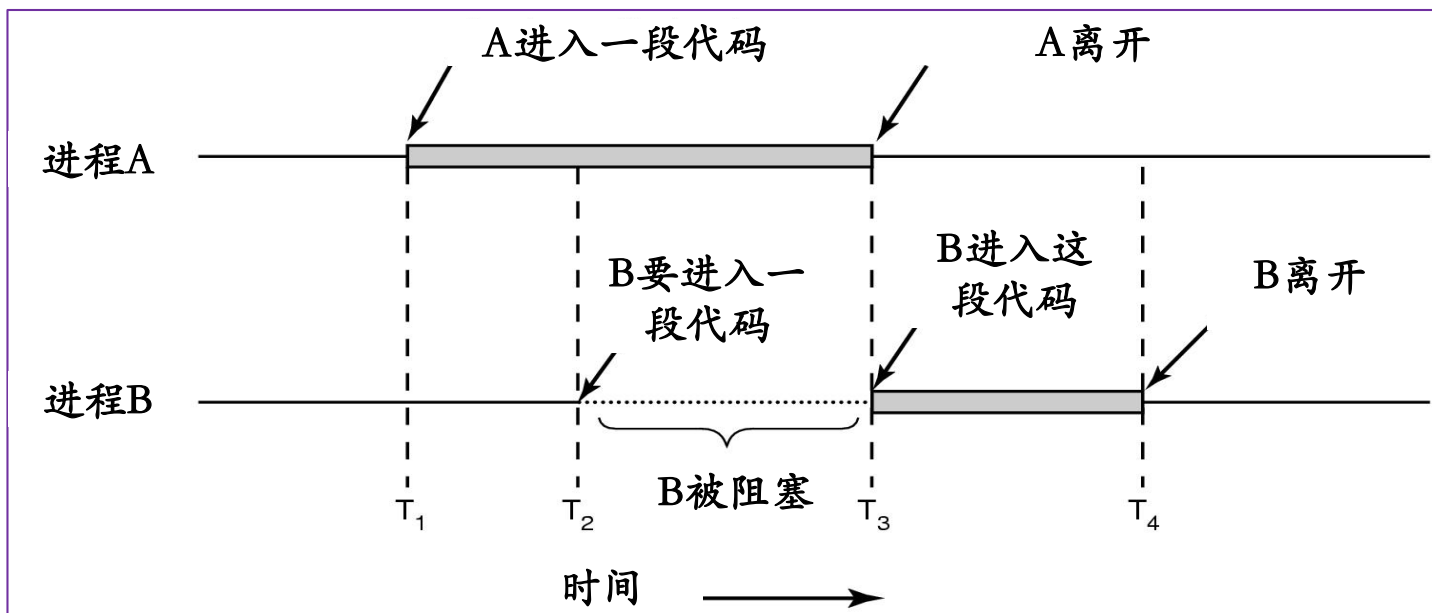
- Some bins get no update, some get 2 (or more!)

进程/线程互斥1

- ▶ 由于各进程要求使用共享资源（变量、文件等），而这些资源需要排他性使用

各进程之间竞争使用这些资源

—— 这一关系称为**进程互斥**



进程/线程互斥2

- ▶ **临界资源: critical resource**

系统中某些资源一次只允许一个进程/线程使用，称这样的资源为**临界资源**或**互斥资源**或**共享变量**

- ▶ **临界区(互斥区): critical section(region)**

各个进程/线程中对某个临界资源（共享变量）实施操作的程序片段

临界区(互斥区)的使用原则

- ▶ **有空让进**: 当没有进程在临界区时, 任何有权使用互斥区的进程可进入临界区
- ▶ **无空等待**: 不允许两个以上的进程同时进入临界区
- ▶ **有限等待**: 任何进入临界区的要求应在有限时间内得到满足
- ▶ **多中择一**: 当没有进程在临界区, 而同时有多个进程要求进入临界区时, 只能让其中之一进入, 其他进程必须等待
- ▶ **让权等待**: 处于等待状态的进程应放弃占用CPU, 以使其他进程有机会得到CPU的使用权

前提: 任何进程无权停止其它进程的运行; 进程之间相对运行速度无硬性规定

进程线程的同步

进程线程同步：synchronization

指系统中多个进程线程中发生的事件存在某种时序关系，需要相互合作，共同完成一项任务

具体地说，一个进程运行到某一点时，要求另一伙伴进程为它提供消息，在未获得消息之前，该进程进入阻塞态，获得消息后被唤醒进入就绪态

实现进程线程互斥的方案

▶ 软件方案

⇒ 其他的互斥软件方案?

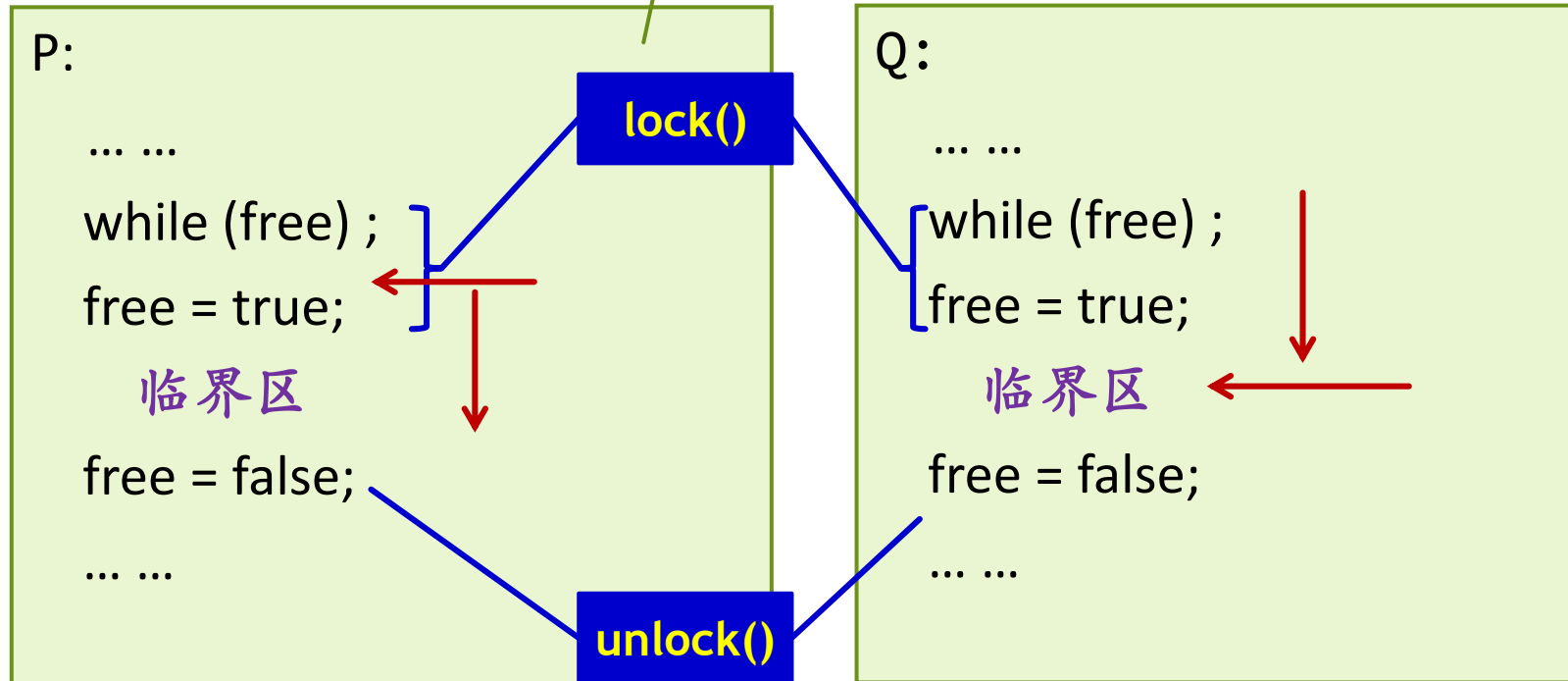
▶ Dekker解法、Peterson解法

▶ 硬件方案

▶ 屏蔽中断、TSL(XCHG)指令

软件解法1

lock()必须满足什么性质?



free: 临界区空闲标志
true: 有进程在临界区
false: 无进程在临界区
初值: free为false

软件解法2

P:

... ..

while (not turn) ;

临界区

turn = false;

... ..

Q:

... ..

while (turn) ;

临界区

turn = true;

... ..

turn: 谁进临界区标志
true: P进程进临界区
false: Q进程进临界区
初值任意

软件解法3

After you 问题

P:

... ..

pturn = true;

while (qturn);

临界区

pturn = false;

... ..

Q:

... ..

qturn = true;

while (pturn);

临界区

qturn = false;

... ..

pturn, qturn: 初值为false

P进入临界区的条件: $pturn \wedge \text{not } qturn$

Q进入临界区的条件: $\text{not } pturn \wedge qturn$

软件解法4——Dekker算法

在解法3基础上
引入turn变量

P:

... ..

pturn = true;

while (qturn) {

if (turn == 2) {

pturn = false;

while (turn == 2);

pturn = true;

}}

临界区

turn = 2;

pturn = false;

... ..

Q:

... ..

qturn = true;

while (pturn) {

if (turn == 1) {

qturn = false;

while (turn == 1);

qturn = true;

}}

临界区

turn = 1;

qturn = false;

... ..

循环

1965年提出
第一个用软件正
确解决互斥问题

软件解法5——Peterson算法

```
#define FALSE 0
#define TRUE 1
#define N      2    // 进程的个数
int turn;           // 轮到谁?
int interested[N];
// 兴趣数组, 初始值均为FALSE

void enter_region ( int process)
    // process = 0 或 1
{
    int other;
    // 另外一个进程的进程号
    other = 1 - process;
    interested[process] = TRUE; //
    表明本进程感兴趣
    turn = process;           // 设置
    标志位
    while( turn == process &&
interested[other] == TRUE);
}
```

循环

```
void leave_region ( int process)
{
    interested[process] = FALSE;
    // 本进程已离开临界区
}
```

进程i:

```
... ..
enter_region ( i );
    临界区
leave_region ( i );
... ..
```

**Peterson算法解决
了互斥访问的问题
，而且克服了强制
轮流法的缺点，可
以完全正常地工作
(1981)**

硬件解法1—关闭中断方法

“开关中断”指令

```
while (true) {  
    /* disable interrupts */;  
    /* critical section */;  
    /* enable interrupts */;  
    /* remainder */;  
}
```

提问：单处理器下，
关闭中断是否符合
临界区的使用原则
(有空让仅、无空
等待、有限等待)？

- 简单，高效
- 代价高，限制CPU并发能力（临界区大小）
- 不适用于多处理器
- 适用于操作系统本身，不适用于用户进程

硬件解法2 — “测试并加锁” 指令

TSL指令：Test and Set Lock

enter_region:

```
TSL REGISTER,LOCK  
CMP REGISTER,#0  
JNE enter_region  
RET
```

循环

| 复制锁到寄存器并将锁置1
| 判断寄存器内容是否是零?
| 若不是零，跳转到enter_region
| 返回调用者，进入了临界区

leave_region:

```
MOVE LOCK,#0  
RET
```

| 在锁中置0
| 返回调用者

硬件解法2 — “测试并加锁” 指令

TSL指令：Test and Set Lock

enter_region:

```
TSL REGISTER, LOCK  
CMP REGISTER, #0  
JNE enter_region  
RET
```

循环

复制锁到寄存器并将锁置1
判断寄存器内容是否是零?
若不是零, 跳转到enter_region
返回调用者, 进入了临界区

leave_region:

```
MOVE LOCK, #0  
RET
```

在锁中置0
返回调用者

思考题：对多处理器
系统有效吗？为什么？

思考题：
XCHG指令

小结

- ▶ 软件方法

 - ▶ 需要编程技巧

- ▶ 硬件方法

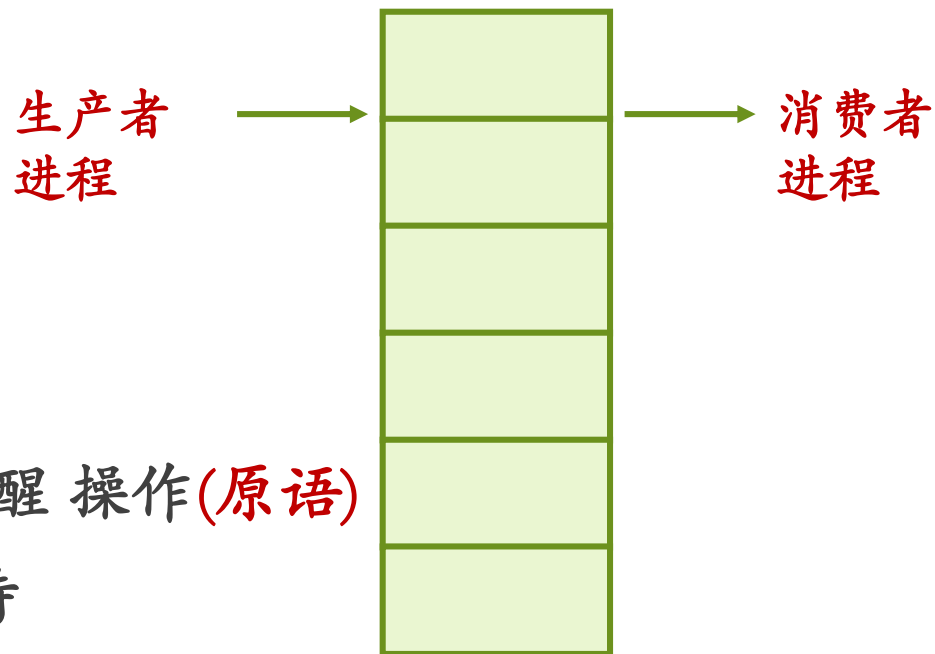
- ▶ 忙等待(busy waiting)

 - 进程在得到临界区访问权之前，持续测试而不做其他事情

 - 自旋锁 Spin lock （多处理器 v）

- ▶ 优先级反转（倒置）

生产者/消费者(有界缓冲区)问题



解决方案

- 提供：睡眠与唤醒操作(原语)
- 要求：避免忙等待

要解决的问题：

- 当缓冲区已满时，生产者不会继续向其中添加数据；
- 当缓冲区为空时，消费者不会从中移走数据

生产者/消费者问题

一种场景：消费者
判断count=0后进入睡眠前被切换

```
#define N 100
int count=0;

void producer(void)
{ int item;
  while(TRUE) {
    item=produce_item();
    if(count==N) sleep();
    insert_item(item);
    count=count+1;
    if(count==1)
      wakeup(consumer);
  }
}
```

检查
count
的值

```
void consumer(void)
{
  int item;
  while(TRUE) {
    if(count==0) sleep();
    item=remove_item();
    count=count-1;
    if(count==N-1)
      wakeup(producer);
    consume_item(item);
  }
}
```

典型的同步机制

- ▶ 信号量及P、V操作
- ▶ 管程
- ▶ 锁
- ▶ 条件变量

信号量及P、V操作

一种经典的进程/线程同步机制

信号量及PV操作

- ▶ 一个特殊变量
- ▶ 用于进程线程间传递信号的一个整数值
- ▶ 定义如下：

```
struct semaphore  
{  
    int count;  
    queueType queue;  
}
```

- ▶ 信号量说明：semaphore **s**;
- ▶ 对信号量可以实施的操作：初始化、P和V (**P、V**分别是荷兰语的**test(proberen)**和**increment(verhogen)**)

一种卓有成效的进程同步机制

由荷兰学者Dijkstra提出，1965年

P、V操作定义

与一些参考书上或
ICS课上定义不同，
但效果是否一样？

P(s)

{

s.count --;

if (s.count < 0)

{

该进程线程状态置为阻塞状态；
将该进程插入相应的等待队列
s.queue 末尾；

重新调度；

}

}

down, semWait

V(s)

{

s.count ++;

if (s.count <= 0)

{

唤醒相应等待队列s.queue中等待
的一个进程线程；
改变其状态为就绪态，并将其插
入就绪队列；

}

}

up, ³⁹semSignal

ICS课的 信号量实现

Reminder: Semaphores

- **Semaphore:** non-negative global integer synchronization variable
- Manipulated by P and V operations:
 - $P(s)$: [while ($s == 0$) wait(); $s--$;]
 - Dutch for "Proberen" (test)
 - $V(s)$: [$s++$;]
 - Dutch for "Verhogen" (increment)
- OS kernel guarantees that operations between brackets [] are executed atomically
 - Only one P or V operation at a time can modify s .
 - When **while** loop in P terminates, only that P can decrement s
- Semaphore invariant: ($s \geq 0$)

有关说明

- P、V操作作为原语操作

原语(primitive or atomic action)

完成某种特定功能的一段程序，具有不可分割性或不可中断性

原语的执行必须是连续的，在执行过程中不允许被中断

可以通过屏蔽中断、测试与设置指令等来实现

- 在信号量上定义了三个操作

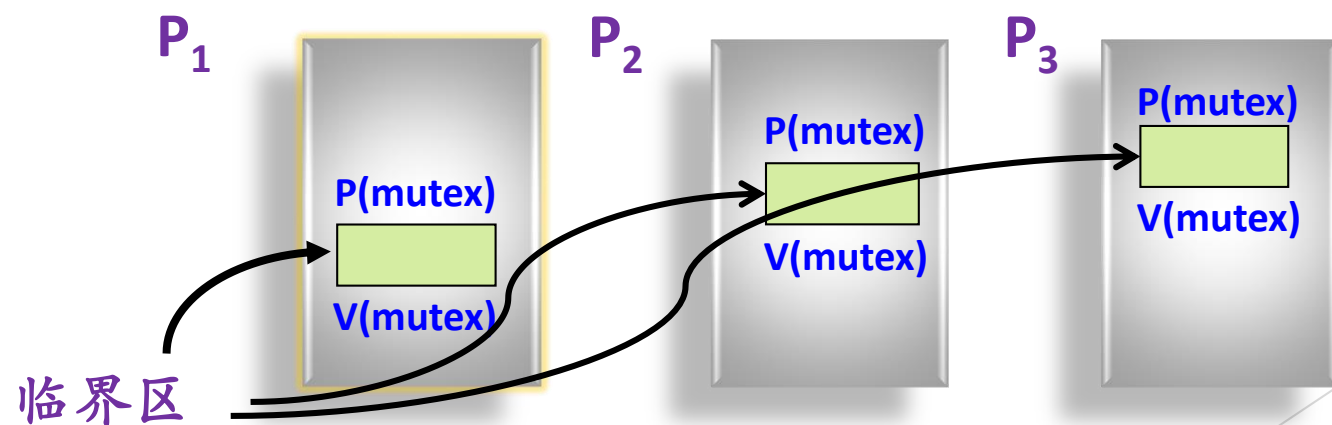
初始化(非负数)、P操作、V操作

- 最初提出的是二元信号量（互斥）

之后推广到一般信号量（多值）或计数信号量（同步）

用PV操作解决进程线程间互斥问题

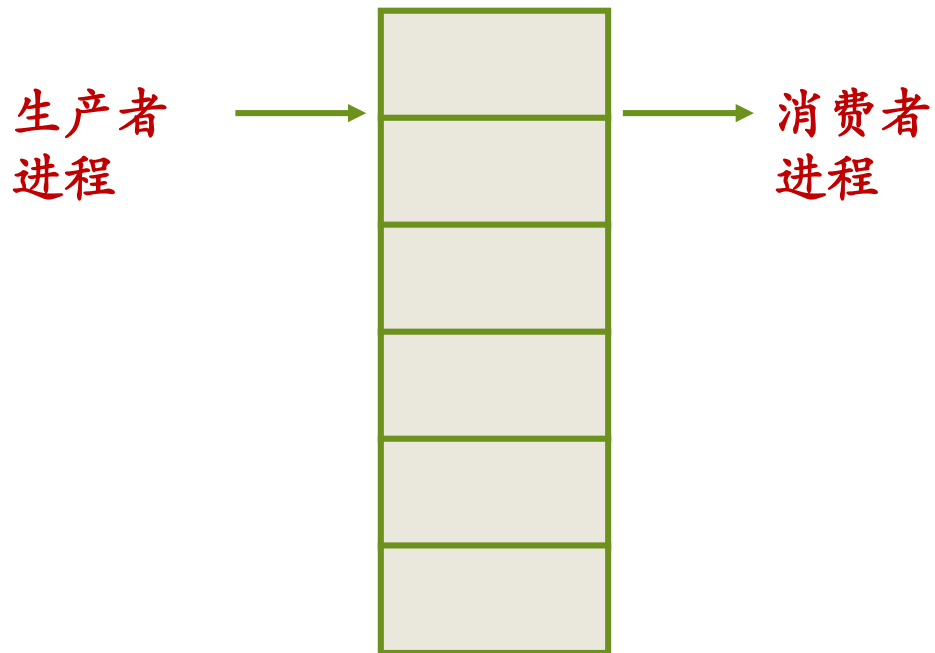
- ▶ 分析并发进程的关键活动，划定临界区
- ▶ 设置信号量 `mutex`，初值为1
- ▶ 在临界区前实施 `P(mutex)`
- ▶ 在临界区之后实施 `V(mutex)`



生产者/消费者(有界缓冲区)问题

问题描述:

- 一个或多个生产者生产某种类型的数据放置在缓冲区中
- 有消费者从缓冲区中取数据, 每次取一项
- 只能有一个生产者或消费者对缓冲区进行操作



要解决的问题:

- 当缓冲区已满时, 生产者不会继续向其中添加数据;
- 当缓冲区为空时, 消费者不会从中移走数据

用信号量解决生产者/消费者问题

```
void producer(void)
{
    int item;
    while(TRUE) {
        item=produce_item();
        P(&empty);
        P(&mutex);
        insert_item(item);
        V(&mutex);
        V(&full);
    }
}
```

```
void consumer(void)
{
    int item;
    while(TRUE) {
        P(&full);
        P(&mutex);
        item=remove_item();
        V(&mutex);
        V(&empty);
        consume_item(item);
    }
}
```

```
#define N 100                /* 缓冲区个数 */
typedef int semaphore; /* 信号量是一种特殊的整型数据 */
semaphore mutex = 1;        /* 互斥信号量：控制对临界区的访问 */
semaphore empty = N;        /* 空缓冲区个数 */
semaphore full = 0;         /* 满缓冲区个数 */
```

讨论一下

```
void producer(void)
{
    int item;
    while(TRUE) {
        item=produce_item();
        P(&empty);
        P(&mutex);
        insert_item(item);
        V(&mutex);
        V(&full);
    }
}
```

位置改变

```
void consumer(void)
{
    int item;
    while(TRUE) {
        P(&full);
        P(&mutex);
        item=remove_item();
        V(&mutex);
        V(&empty);
        consume_item(item);
    }
}
```

位置改变

若颠倒两个P操作的顺序?

若颠倒两个V操作的顺序?

用信号量解决读者-写者问题

问题描述：

多个进程共享一个数据区，这些进程分为两组：

读者进程：只读数据区中的数据 ○

写者进程：只往数据区写数据 ○

要求满足条件：

- ✓ 允许多个读者同时执行读操作
- ✓ 不允许多个写者同时操作
- ✓ 不允许读者、写者同时操作

给出一种
应用场景

第一类：读者优先

如果读者执行：

- 无其他读者、写者，该读者可以读
- 若已有写者等，但有其他读者正在读，则该读者也可以读
- 若有写者正在写，该读者必须等

如果写者执行：

- 无其他读者、写者，该写者可以写
- 若有读者正在读，该写者等待
- 若有其他写者正在写，该写者等待

第一类读者-写者问题的解法

```
void reader(void)
```

```
{
```

```
    while (TRUE) {
```

```
        .....
```

```
        P(w);
```

不需要每个
读者都做

读操作

```
        V(w);
```

```
        .....
```

```
    }
```

```
}
```

```
void writer(void)
```

```
{
```

```
    while (TRUE) {
```

```
        .....
```

```
        P(w);
```

写操作

```
        V(w);
```

```
        .....
```

```
    }
```

```
}
```

第一类读者-写者问题的解法

Linux: read_lock,
write_lock

```
void reader(void)
```

```
{
```

```
while (TRUE) {
```

```
  P(mutex);
```

```
  rc = rc + 1;
```

```
  if (rc == 1) P(w);
```

```
  V(mutex);
```

读操作

```
  P(mutex);
```

```
  rc = rc - 1;
```

```
  if (rc == 0) V(w);
```

```
  V(mutex);
```

其他操作

```
}
```

第一个读者

最后一个读者

临界区

```
void writer(void)
```

```
{
```

```
while (TRUE) {
```

```
  .....
```

```
  P(w);
```

写操作

```
  V(w);
```

```
}
```

```
}
```

管程

语言机制、条件变量/*wait/signal*

为什么引入管程？

问题

- 信号量机制的不足：程序编写困难、效率低

解决

- Brinch Hansen(1973)
- Hoare (1974)

方案

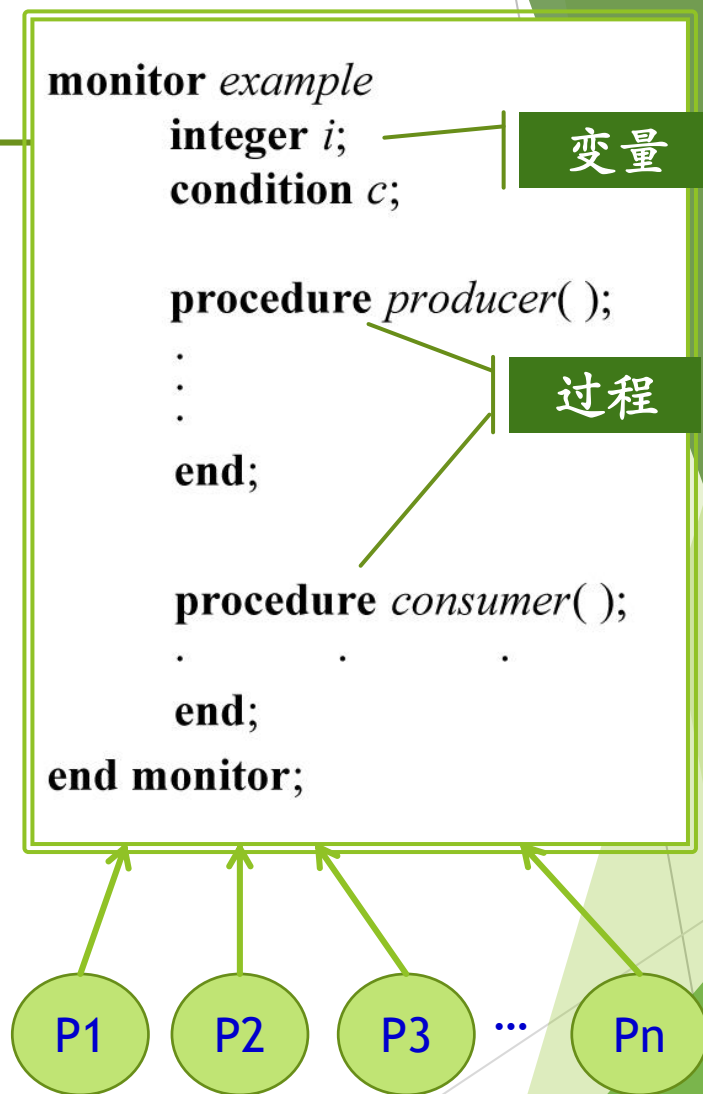
- 在程序设计语言中引入管程成分
- 一种高级同步机制

管程的定义

- ▶ 是一个特殊的模块
- ▶ 有一个名字
- ▶ 由关于**共享资源的数据结构**及在其上操作的一组**过程**组成

□ 进程与管程

进程只能通过调用**管程中的过程**来间接访问管程中的数据结构



管程要保证什么？

► 作为一种同步机制，管程要解决两个问题？

► 互斥

管程是互斥进入的

——为了保证管程中数据结构的数据完整性

注意：管程的互斥性是由编译器负责、保证的

► 同步

管程中设置条件变量及等待/唤醒操作以解决同步问题

可以让一个进程或线程等待在条件变量上等待（此时，应先释放管程的使用权），也可以通过发送信号将等待在条件变量上的进程或线程唤醒

使用管程时遇到的问题

是否会出现这样一种情况，有多个进程同时在管程中 ?

场景：

当一个进入管程的进程执行等待操作时，它应当释放管程的互斥权

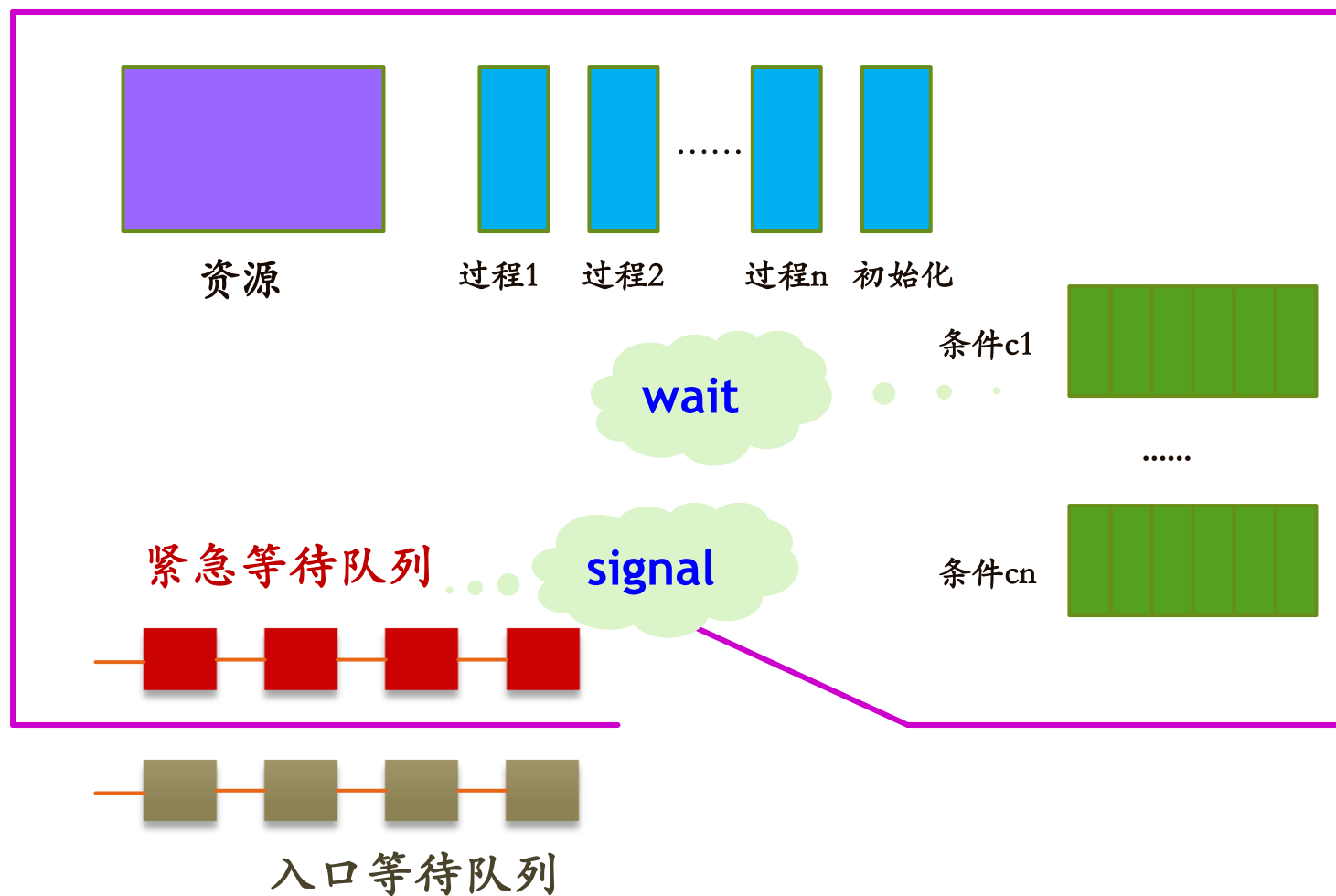
当后面进入管程的进程执行唤醒操作时（例如，P唤醒Q），管程中便存在两个同时处于活动状态的进程

如何解决？

处理方法：

- P等待Q执行 (Hoare) v
- Q等待P继续执行
- 规定唤醒为管程中最后一个可执行的操作 (Hansen, 并发pascal)
- 由系统决定 MESA

Hoare管程示意图



➤ 管程是互斥进入的，当一个进程试图进入一个已被占用的管程时，应当在管程的入口处等待

● 为此，管程的入口处设置一个进程等待队列，称作入口等待队列

➤ 如果进程P唤醒进程Q，则P等待Q执行；如果进程Q执行中又唤醒进程R，则Q等待R执行；.....，如此，在管程内部可能会出现多个等待进程

● 管程内需要设置一个进程等待队列，称为紧急等待队列，其优先级高于入口等待队列的优先级

Hoare管程 — 条件变量的实现

- 条件变量——在管程内部说明和使用的一种特殊类型的变量
- `var c:condition;`
- 对于条件型变量，可以执行wait和signal操作

wait(c):

如果紧急等待队列非空，则唤醒第一个等待者，否则释放管程的互斥权；
执行此操作的进程进入c链尾部

signal(c):

如果c链为空，则相当于空操作，执行此操作的进程继续执行；否则唤醒第一个等待者，执行此操作的进程进入紧急等待队列的尾部

用管程解决生产者消费者问题

注：按照最早的Pascal语义，
procedure表示没有返回值的
过程，
function表示有返回值的
过程

```
monitor ProducerConsumer
  condition full, empty;
  integer count;

  procedure insert (item: integer);
  begin
    if count == N then wait(full);
    insert_item(item); count++;
    if count == 1 then signal(empty);
  end;

  function remove: integer;
  begin
    if count == 0 then wait(empty);
    remove = remove_item; count--;
    if count == N-1 then signal(full);
  end;

  count:=0;
end monitor;
```

```
procedure producer;
begin
  while true do
  begin
    item = produce_item;
    ProducerConsumer.insert(item);
  end
end;

procedure consumer;
begin
  while true do
  begin
    item=ProducerConsumer.remove;
    consume_item(item);

  end
end;
```

管程的实现

管程实现的两个主要途径：

- * 直接构造 → 效率高

- * 间接构造

→ 用某种已经实现的同步机制去构造

例如：用信号量
及P、V操作构造
管程

直接构造管程

The monitor is a programming-language construct that provides equivalent functionality to that of semaphores and that is easier to control.

管程：抽象数据类型

有一个明确定义的操作集合，通过它且只有通过它才能操纵该数据类型的实例

实现管程结构必须保证两点：

- (1) 只能通过管程的某个过程才能访问资源；
- (2) 过程是互斥的，某个时刻只能有一个进程或线程驻留在给定的管程中

条件变量：为提供进程与其他进程通信或同步而引入

► wait/signal

Java中的管程(1/2)

```
public class ProducerConsumer {  
    static final int N = 100; // constant giving the buffer size  
    static producer p = new producer(); // instantiate a new producer thread  
    static consumer c = new consumer(); // instantiate a new consumer thread  
    static our_monitor mon = new our_monitor(); // instantiate a new monitor  
  
    public static void main(String args[]) {  
        p.start(); // start the producer thread  
        c.start(); // start the consumer thread  
    }  
  
    static class producer extends Thread {  
        public void run() { // run method contains the thread code  
            int item;  
            while (true) { // producer loop  
                item = produce_item();  
                mon.insert(item);  
            }  
        }  
        private int produce_item() { ... } // actually produce  
    }  
}
```

Java中的管程(2/2)

```
static class consumer extends Thread {
    public void run() {run method contains the thread code
        int item;
        while (true) {    // consumer loop
            item = mon.remove();
            consume_item (item);
        }
    }
    private void consume_item(int item) { ... } // actually consume
}

static class our_monitor { // this is a monitor
    private int buffer[] = new int[N];
    private int count = 0, lo = 0, hi = 0; // counters and indices
    public synchronized void insert(int val) {
        if (count == N) go_to_sleep(); // if the buffer is full, go to sleep
        buffer [hi] = val; // insert an item into the buffer
        hi = (hi + 1) % N; // slot to place next item in
        count = count + 1; // one more item in the buffer now
        if (count == 1) notify(); // if consumer was sleeping, wake it up
    }
}
```

用信号量机制构造等价的 管程机制

间接构造管程

首先，定义一个
一个基础管程

```
TYPE one_instance=RECORD
  mutex: semaphore; (初值1)
  urgent: semaphore; (初值0)
  urgent_count: integer; (初值0)
END;
```

```
TYPE monitor_one =MODULE;
define enter,leave,wait,signal;
```

mutex (入口互斥队列)
urgent (紧急等待队列)
urgent_count (紧急等待队列计数)

```
PROCEDURE enter(VAR
  instance:one_instance);
BEGIN
  P(instance.mutex)
END;
```

```
PROCEDURE leave(VAR
  instance:one_instance);
BEGIN
  IF instance.urgent_count > 0
  THEN
    BEGIN instance.urgent_count--;
      V(instance.urgent)
    END
  ELSE
    V(instance.mutex)
  END;
```

基础管程中的wait和signal操作

PROCEDURE wait

```
(VAR instance:one_instance;VAR
s:semaphore;VAR count:integer);
BEGIN
    count++;
    IF instance.urgent_count>0 THEN
        BEGIN
            instance.urgent_count--;
            V(instance.urgent)
        END
    ELSE
        V(instance.mutex);
    P(s);
END;
```

PROCEDURE signal

```
(VAR instance:one_instance;
VAR s:semaphore;VAR
count:integer);

BEGIN
    IF count>0 THEN
        BEGIN
            count--;
            instance.urgent_count++;
            V(s);
            P(instance.urgent)
        END
    END;
```

用构造的管程解决读者-写者问题

```
TYPE r_and_w=MODULE;  
VAR  
instance: one_instance;  
rq, wq: semaphore;  
r_count, w_count: integer;  
reading_count, write_count:  
integer;  
  
define  
    start_r, finish_r, start_w,  
    finish_w;  
use monitor_one.enter,  
    monitor_one.leave,  
    monitor_one.wait,  
    monitor_one.signal;
```

```
BEGIN  
    reading_count:=0;  
    write_count:=0;  
    r_count:=0;  
    w_count:=0;  
END;
```

读者的活动:

```
    r_and_w.start_r;  
    读操作;  
    r_and_w.finish_r;
```

写者的活动:

```
    r_and_w.start_w;  
    写操作;  
    r_and_w.finish_w;
```

start_r 和 finish_r的实现

```
PROCEDURE start_r;  
BEGIN  
    monitor_one.enter(instance);  
    IF write_count>0 THEN  
        monitor_one.wait  
            (instance,rq,r_count);  
    reading_count++;  
    monitor_one.signal  
        (instance,rq,r_count);  
    monitor_one.leave(instance);  
END;
```

```
PROCEDURE finish_r;  
BEGIN  
    monitor_one.enter(instance);  
    reading_count--;  
    IF reading_count=0 THEN  
        monitor_one.signal (instance,wq,w_count);  
    monitor_one.leave(instance);  
END;
```

start_w 和 finish_w的实现

```
PROCEDURE start_w;  
BEGIN  
    monitor_one.enter(instance);  
    write_count++;  
    IF (write_count>1)  
        OR(reading_count>0)  
    THEN monitor_one.wait  
        (instance,wq,w_count);  
    monitor_one.leave(instance);  
END;
```

```
PROCEDURE finish_w;  
BEGIN  
    monitor_one.enter(instance);  
    write_count --;  
    IF (r_count=0) THEN  
        monitor_one.signal(instance,wq,w_count);  
    ELSE  
        monitor_one.signal (instance,rq,r_count);  
    monitor_one.leave(instance);  
END
```

各种机制的层次

应用

并发进程/线程

系统

同步机制
(信号量、管程、锁、条件变量……)

硬件

原子操作
(load/store、开关中断、测试&设置、……)

管程讨论

MESA 管程

为什么有不同的管程 ?

解决问题场景：有多个进程同时在管程中

► 如何解决避免管程中同时有两个活跃进程？

确定signal之后的规则

► 三种方案

► Hoare管程

► Concurrent Pascal (Hansen)

► Mesa管程

- 在管程设计时就确定了谁将有优先权，不灵活
- 发挥操作系统的作用

Mesa管程

- Hoare管程：signal的缺陷
 - ✓ 两次额外的进程切换
 - ✓ 是否会使条件队列中的进程永久挂起？
- Lampson和Redell，Mesa语言（1980）
- 解决：
 - ✓ **signal → notify**
 - ✓ **notify**：当一个正在管程中的进程执行notify(x)时，它使得x条件队列得到通知，发信号的进程继续执行

使用notify要注意的问题

- ▶ **notify的结果:**

位于条件队列头的进程在将来合适的时候且当处理器可用时恢复执行

- ▶ 由于不能保证在它之前没有其他进程进入管程，因而这个进程必须重新检查条件

→ 用**while**循环取代**if**语句

- ▶ 导致对条件变量至少多一次额外的检测（但不再有额外的进程切换），并且对等待进程在**notify**之后何时运行没有任何限制

Mesa管程：生产者-消费者问题

```
void append (char x)
{
    while(count == N) cwait(notfull);    /* buffer is full; avoid overflow */
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;                             /* one more item in buffer */
    cnotify(notempty);                   /* notify any waiting consumer */
}

void take (char x)
{
    while(count == 0) cwait(notempty); /* buffer is empty; avoid underflow */
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;                             /* one fewer item in buffer */
    cnotify(notfull);                   /* notify any waiting producer */
}
```

William Stallings

改进notify

- ▶ 对notify的一个很有用的改进

- ▶ 给每个条件原语关联一个监视计时器，不论条件是否被通知，一个等待时间超时的进程将被设置为就绪状态
- ▶ 当该进程被调度执行时，会再次检查相关条件，如果条件满足则继续执行

- ▶ 超时可以防止如下情况的发生：

当某些进程在产生相关条件的信号之前失败时，等待该条件的进程被无限制地推迟执行而处于饥饿状态

broadcast原语的引入

broadcast:

可以使所有在该条件上等待的进程都进入就绪状态

- ▶ 当一个进程不知道有多少进程将被激活时，这种方式是很方便的
 - ▶ 例如，在生产者/消费者问题中，假设insert和remove函数都适用于可变长度的字符块，此时，如果一个生产者往缓冲区中添加了一批字符，它不需要知道每个正在等待的消费者准备消耗多少字符，而仅仅产生一个broadcast，所有正在等待的进程都得到通知并再次尝试运行
 - ▶ 当一个进程难以准确判定将激活哪个进程时，也可使用广播

Hoare管程与Mesa管程的比较

- ✓ Mesa管程优于Hoare管程之处在于应用Mesa管程时出错比较少

在Mesa管程中，由于每个过程在收到信号后都检查管程变量，并且由于使用了while结构，一个进程不正确的广播或发信号，不会导致收到信号的程序出错

收到信号的程序将检查相关的变量，如果期望的条件没有满足，它会重新继续等待

PTHREADS

锁、条件变量

锁 (互斥量 mutex)

- ▶ 一种容易实现且有效的机制
- ▶ 适用于管理共享资源或一小段代码
- ▶ 常用于实现 **用户空间线程包**
- ▶ 锁（互斥量）处于两种状态：
加锁 或 **解锁**
常为“锁”为一个整型量 → **0为解锁/1为加锁**
- ▶ 提供两个过程：mutex_lock和mutex_unlock
(类似地，lock()和unlock() / acquire()和release())

```
acquire(lock)
...
critical section
...
release(lock)
```

1. 定义互斥锁类型，包括一个表示锁状态的布尔变量 (locked) 和一个等待锁的线程队列 (queue)。

```
struct mutex {  
    bool locked;  
    queue_t queue;  
};
```

2. 初始化锁为未锁定状态。

```
void mutex_init(mutex_t *mutex) {  
    mutex->locked = false;  
    init_queue(&mutex->queue);  
}
```

3. 加锁操作，当锁已经被其他线程持有时，当前线程将阻塞并加入到等待队列中。

```
void mutex_lock(mutex_t *mutex) {  
    while (atomic_swap(&mutex->locked, true) == true) {  
        // 等待锁释放  
        add_to_queue(&mutex->queue, current_thread);  
        yield_cpu(); // 让出CPU时间片  
    }  
}
```

4. 解锁操作，当锁被释放后，如果有等待线程则唤醒其中一个线程继续执行。

```
void mutex_unlock(mutex_t *mutex) {  
    atomic_swap(&mutex->locked, false);  
    if (!is_queue_empty(&mutex->queue)) {  
        // 唤醒等待的线程  
        thread_t *next_thread = remove_from_queue(&mutex->queue);  
        wakeup_thread(next_thread);  
    }  
}
```

锁的实现伪码—ChatGPT

Pthreads中的同步机制

Thread call	Description
Pthread_mutex_init	Create a mutex
Pthread_mutex_destroy	Destroy an existing mutex
Pthread_mutex_lock	Acquire a lock or block
Pthread_mutex_trylock	Acquire a lock or fail
Pthread_mutex_unlock	Release a lock

互斥量

Pthreads中保护临界区

条件变量

Pthreads中的
同步机制

Thread call	Description
Pthread_cond_init	Create a condition variable
Pthread_cond_destroy	Destroy a condition variable
Pthread_cond_wait	Block waiting for a signal
Pthread_cond_signal	Signal another thread and wake it up
Pthread_cond_broadcast	Signal multiple threads and wake all of them

用Pthreads解决生产者-消费者问题

```
int main(int argc, char **argv)
{
    pthread_t pro, con;
    pthread_mutex_init(&the_mutex, 0);
    pthread_cond_init(&condc, 0);
    pthread_cond_init(&condp, 0);
    pthread_create(&con, 0, consumer, 0);
    pthread_create(&pro, 0, producer, 0);
    pthread_join(pro, 0);
    pthread_join(con, 0);
    pthread_cond_destroy(&condc);
    pthread_cond_destroy(&condp);
    pthread_mutex_destroy(&the_mutex);
}
```

Pthread_mutex

- ▶ 一个基础的使用mutex保护共享变量balance的场景
 - ▶ 来自OSTEP Ch28

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3 Pthread_mutex_lock(&lock); // wrapper; exits on failure
4 balance = balance + 1;
5 Pthread_mutex_unlock(&lock);
```

Pthre

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000                                /* how many numbers to produce */

pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;                        /* buffer used between producer and consumer */
int buffer = 0;                                       /* produce data */

void *producer(void *ptr)
{
    int i;

    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer != 0) pthread_cond_wait(&condp, &the_mutex);
        buffer = i; /* put item in buffer */
        pthread_cond_signal(&condc); /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

void *consumer(void *ptr)                                /* consume data */
{
    int i;

    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer == 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = 0; /* take item out of buffer */
        pthread_cond_signal(&condp); /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}
```

一个缓冲区

讨论: pthread_cond_wait

- ▶ 当发起一个pthread_cond_wait之后，分解为三个动作：
 1. 解锁
 2. 等待，当收到一个解除等待的信号（pthread_cond_signal或者pthread_cond_broadcast）之后，pthread_cond_wait 马上要做的动作是：
 3. 上锁

Pthre

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000                                /* how many numbers to produce */

pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;
int buffer = 0;                                       /* buffer used between producer and consumer */

void *producer(void *ptr)                            /* produce data */
{
    int i;
    for (i= 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer != 0) pthread_cond_wait(&condp, &the_mutex);
        buffer = i; /* put item in buffer */
        pthread_cond_signal(&condc); /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

void *consumer(void *ptr)                            /* consume data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer == 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = 0; /* take item out of buffer */
        pthread_cond_signal(&condp); /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}
```

一个缓冲
区

Pthre

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000                                /* how many numbers to produce */

pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;
int buffer = 0;                                       /* buffer used between producer and consumer */

void *producer(void *ptr)                             /* produce data */
{
    int i;
    for (i= 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        if (buffer != 0) pthread_cond_wait(&condp, &the_mutex);
        buffer = i;                               /* put item in buffer */
        pthread_cond_signal(&condc);             /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

void *consumer(void *ptr)                             /* consume data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        if (buffer == 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = 0;                               /* take item out of buffer */
        pthread_cond_signal(&condp);             /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}
```

一个缓冲区

各种机制的层次

应用

并发进程/线程

系统

同步机制
(信号量、管程、锁、条件变量……)

硬件

原子操作
(load/store、开关中断、测试&设置、……)

锁的实现

MUTEX

锁（互斥量mutex）的实现

► 通过开关中断指令

```
lock() {  
    disable interrupts  
    while (value != FREE) ;  
    value = BUSY  
    enable interrupts  
}
```

- 等待锁变为空闲
- 获得锁

while改成if可能会导致错误唤醒，例如consumer被唤醒，但是buffer依然为0，producer还没生产成功

思考：这样实现锁是否正确？

一种修订

```
lock() {  
    disable interrupts  
    while (value != FREE) {  
        enable interrupts  
        disable interrupts  
    }  
    value = BUSY  
    enable interrupts  
}
```



设计一个方案
编码验证一下

另一种方案

无忙等待
+
开关中断

```
lock() {  
    disable interrupts  
    if (value == FREE) {  
        value = BUSY  
    } else {  
        add thread to queue of threads waiting for this lock  
        switch to next run-able thread  
    }  
    enable interrupts  
}
```

← ①

← ②

← ③

需要大家思考
的问题：在①
或②或③处开
中断是否可行？
为什么？

思考一下

B是目前持有锁的线程

Thread A

```
enable interrupts
}  
<user code runs>  
lock() {  
    disable interrupts  
    ...  
    switch
```

```
    back from switch  
    enable interrupts  
}
```

Thread B

```
yield() {  
    disable interrupts  
    switch
```

```
    back from switch  
    enable interrupts  
}  
<user code runs>  
unlock() (move thread A to ready queue)  
yield() {  
    disable interrupts  
    switch
```

通过Test&Set指令实现锁

mutex_lock:

TSL REGISTER,MUTEX

CMP REGISTER,#0

JZE ok

CALL thread_yield

JMP mutex_lock

ok:

RET

| copy mutex to register and set mutex to 1

| was mutex zero?

| if it was zero, mutex was unlocked, so return

| mutex is busy; schedule another thread

| try again

| return to caller; critical region entered

为什么?

mutex_unlock:

MOVE MUTEX,#0

RET

| store a 0 in mutex

| return to caller

用户空间实现锁，如线程包

对比

硬件解法2 —“测试并加锁” 指令

TSL指令：Test and Set Lock

enter_region:

TSL REGISTER, LOCK

CMP REGISTER, #0

JNE enter_region

RET

| 复制锁到寄存器并将锁置1
| 判断寄存器内容是否为零?
| 若不是零, 跳转到enter_region
| 返回调用者, 进入了临界区

循环

leave_region:

MOVE LOCK, #0

RET

| 在锁中置0
| 返回调用者

XV6 并发同步机制

实例

XV6: 锁

```
#include<kernel/spinlock.h>
#include<kernel/sleeplock.h>
```

► spinlock, sleeplock

- spinlock: 持有锁的cpu, 锁的名字, 锁定状态
- sleeplock: 使用自旋锁保护内部结构, 记录持有锁的pid

► console, proc, log, kmem, ...

```
1 // Mutual exclusion lock.
2 ✓ struct spinlock {
3     uint locked;           // Is the lock held?
4
5     // For debugging:
6     char *name;            // Name of lock.
7     struct cpu *cpu;       // The cpu holding the lock.
8 };
```

```
1 // Long-term locks for processes
2 ✓ struct sleeplock {
3     uint locked;           // Is the lock held?
4     struct spinlock lk;    // spinlock protecting this sleep lock
5
6     // For debugging:
7     char *name;            // Name of lock.
8     int pid;               // Process holding lock
9 };
```

XV6: 锁的运用

► spinlock在保护关键数据结构时使用

- proc: allocproc, allocpid
- kmem: kalloc, kfree
- log: begin_op, end_op
- console: consoleread, consoleintr

```
84 // Per-process state
85 ✓ struct proc {
86     struct spinlock lock;
87
88     // p->lock must be held when using these:
89     enum procstate state; // Process state
90     void *chan;           // If non-zero, sle
91     int killed;           // If non-zero, hav
92     int xstate;           // Exit status to b
93     int pid;              // Process ID
```

```
40 ✓ struct log {
41     struct spinlock lock;
42     int start;
43     int size;
44     int outstanding; // how many FS sys calls are execut
45     int committing; // in commit(), please wait.
46     int dev;
47     struct logheader lh;
48 };
49 struct log log;
```

```
105 // Look in the process table for an UNUSED proc.
106 // If found, initialize state required to run in the kernel,
107 // and return with p->lock held.
108 // If there are no free procs, or a memory allocation fails, return 0.
109 static struct proc*
110 allocproc(void)
111 {
112     struct proc *p;
113
114     for(p = proc; p < &proc[NPROC]; p++) {
115         acquire(&p->lock);
116         if(p->state == UNUSED) {
117             goto found;
118         } else {
119             release(&p->lock);
120         }
121     }
122     return 0;
```

```
21 struct {
22     struct spinlock lock;
23     struct run *freelist;
24 } kmem;
```

```
44 struct {
45     struct spinlock lock;
46
47     // input
48 #define INPUT_BUF_SIZE 128
49     char buf[INPUT_BUF_SIZE];
50     uint r; // Read index
51     uint w; // Write index
52     uint e; // Edit index
53 } cons;
```

XV6: 锁的应用 (续)

- ▶ `acquiresleep`获取睡眠锁，`releasesleep`释放睡眠锁，`holdingsleep`查询当前进程是否持有当前sleeplock
- ▶ 对inode的操作使用了sleeplock，参考fs.c, bio.c文件

```
21 void
22 acquiresleep(struct sleeplock *lk)
23 {
24     acquire(&lk->lk);
25     while (lk->locked) {
26         sleep(lk, &lk->lk);
27     }
28     lk->locked = 1;
29     lk->pid = myproc()->pid;
30     release(&lk->lk);
31 }
```

```
33 void
34 releasesleep(struct sleeplock *lk)
35 {
36     acquire(&lk->lk);
37     lk->locked = 0;
38     lk->pid = 0;
39     wakeup(lk);
40     release(&lk->lk);
41 }
```

```
43 int
44 holdingsleep(struct sleeplock *lk)
45 {
46     int r;
47
48     acquire(&lk->lk);
49     r = lk->locked && (lk->pid == myproc()->pid);
50     release(&lk->lk);
51     return r;
52 }
```

XV6: PV操作与信号量

- ▶ XV6里没有直接提供Semaphore信号量，但是部分内核模块封装了相似语义的模拟
- ▶ 回忆：前一页上面sleeplock的实现——等待与唤醒
- ▶ 应用：proc, bio, pipe, ...
- ▶ sleep(chan, lock)对应P操作，wakeup(chan)对应V操作
 - ▶ chan：信号传递机制，会记录在进程pcb内使对应进程被唤醒
- ▶ sleep等待时会释放持有的lock，防止死锁

XV6: sleep与wakeup

```
545 // Atomically release lock and sleep on chan.
546 // Reacquires lock when awakened.
547 void
548 ... sleep(void *chan, struct spinlock *lk)
549 {
550     struct proc *p = myproc();
551
552     // Must acquire p->lock in order to
553     // change p->state and then call sched.
554     // Once we hold p->lock, we can be
555     // guaranteed that we won't miss any wakeup
556     // (wakeup locks p->lock),
557     // so it's okay to release lk.
558
559     acquire(&p->lock); //DOC: sleeplock1
560     release(lk);      释放当前持有的lock
561
562     // Go to sleep.
563     p->chan = chan;
564     p->state = SLEEPING;
565
566     sched();          触发调度, 让出CPU
567
568     // Tidy up.
569     p->chan = 0;
570
571     // Reacquire original lock.
572     release(&p->lock);
573     acquire(lk);      被唤醒时又会持有lk
574 }
```

```
576 // Wake up all processes sleeping on chan.
577 // Must be called without any p->lock.
578 void
579 wakeup(void *chan)
580 {
581     struct proc *p;
582
583     for(p = proc; p < &proc[NPROC]; p++) {
584         if(p != myproc()){
585             acquire(&p->lock);
586             if(p->state == SLEEPING && p->chan == chan) {
587                 p->state = RUNNABLE;
588             }
589             release(&p->lock);
590         }
591     }
592 }
```

XV6: 生产者-消费者问题

#include<kernel/pipe.c>

► pipe: pi->data是环形缓冲区（有限大小）

```
76  int
77  pipewrite(struct pipe *pi, uint64 addr, int n)
78  {
79      int i = 0;
80      struct proc *pr = myproc();
81
82      acquire(&pi->lock);
83      while(i < n){
84          if(pi->readopen == 0 || killed(pr)){
85              release(&pi->lock);
86              return -1;
87          }
88          if(pi->nwrite == pi->nread + PIPESIZE){ //DOC: pipewrite-full
89              wakeup(&pi->nread);
90              sleep(&pi->nwrite, &pi->lock);
91          } else {
92              char ch;
93              if(copyin(pr->pagetable, &ch, addr + i, 1) == -1)
94                  break;
95              pi->data[pi->nwrite++ % PIPESIZE] = ch;
96              i++;
97          }
98      }
99      wakeup(&pi->nread);
100     release(&pi->lock);
101
102     return i;
103 }
```

```
105  int
106  piperead(struct pipe *pi, uint64 addr, int n)
107  {
108      int i;
109      struct proc *pr = myproc();
110      char ch;
111
112      acquire(&pi->lock);
113      while(pi->nread == pi->nwrite && pi->writeopen){ //DOC: pipe-empty
114          if(killed(pr)){
115              release(&pi->lock);
116              return -1;
117          }
118          sleep(&pi->nread, &pi->lock); //DOC: piperead-sleep
119      }
120      for(i = 0; i < n; i++){ //DOC: piperead-copy
121          if(pi->nread == pi->nwrite)
122              break;
123          ch = pi->data[pi->nread++ % PIPESIZE];
124          if(copyout(pr->pagetable, addr + i, &ch, 1) == -1)
125              break;
126      }
127      wakeup(&pi->nwrite); //DOC: piperead-wakeup
128      release(&pi->lock);
129      return i;
130 }
```

总结

- ▶ 并发：竞态race，同步与互斥
- ▶ 信号量及P-V操作
- ▶ 生产者-消费者问题；读者-写者问题
- ▶ 管程
- ▶ Pthreads
- ▶ Mutex，锁机制

Many Thanks

The End